Proprietary

MOBILE APPLICATION AND SERVICE DEVELOPMENT

Lecture 2

Introduction to Mobile Computing · Introduction to Kotlin (Part 1)

Academic year 2026/27 · Faculty of Electrical Engineering, University of Tuzla

Proprietary

TODAY'S PLAN

Context first, then code

PART 1 · ~60 MIN

Introduction to mobile computing

- History and evolution of devices
- 3-tier architecture
- Mobile ecosystem and dev environments
- Development approaches and factors
- New dimensions of mobile computing

PART 2 · ~60 MIN

Introduction to Kotlin (Part 1)

- Variables, types, strings
- Conditions, loops, collections
- Functions
- Null safety and lambdas
- Exercise: reading and reviewing code

Proprietary

Why do we build mobile apps?

60%

of global web traffic comes from mobile devices

60.04% mobile vs 39.96% desktop · September 2026 · source: StatCounter Global Stats

Question: in September 2024 it was 63.38%. Why might the share have dropped?

Proprietary

KEY CONCEPTS

What is mobile computing?

Using computing resources and services **on the move** — without a fixed location or a fixed network connection. Mobile app development is building software for smartphones, tablets and other mobile devices.

Hardware

The device: touchscreen, sensors, camera, GPS, battery

Communication

The network: 4G/5G, Wi-Fi, Bluetooth, NFC

Software

OS, apps and cloud services

Proprietary

MARKET · SEPTEMBER 2026

Android dominates, especially here

WORLDWIDE

69%

Android · iOS 31%

BOSNIA AND HERZEGOVINA

76%

Android · iOS 24%

Phones are used for everything: communication, entertainment, shopping, banking, work, learning — and taking attendance.

Source: StatCounter Global Stats, Mobile OS Market Share, Sept 2026 (Worldwide: 69.17 / 30.81 · BiH: 75.67 / 24.32)

EVOLUTION OF DEVICES

Five eras of mobile phones

1983 →

Brick

Big, heavy, calls only. Motorola DynaTAC.

1990s

Candy bar

Smaller, SMS, first games. Nokia.

2000s

Feature

Camera, MP3, WAP, flip designs.

~2002 →

Smartphone

Keyboard, e-mail, PDA. BlackBerry, Palm Treo.

2007 →

Touch

Multi-touch, app stores, no buttons. iPhone, Android.

Today: foldables, tablets, watches and other wearables — one app, many screen formats.

Proprietary

QUIZ

The first commercial mobile phone: Motorola DynaTAC 8000X

Year?

???

Weight?

???

Size?

???

Price?

???



Proprietary

ANSWER

The first commercial mobile phone: Motorola DynaTAC 8000X

Year

1983

Weight

790 g

Size

44 × 330 × 89 mm

Price

$3,995

+ monthly subscription + per-minute cha



QUICK OVERVIEW · 1983–2002

From phone to “computer in your pocket”

1983

Motorola introduces the first commercial mobile phone.

1994

IBM Simon — the first “smart” phone: touchscreen, fax and e-mail.

1997

Nokia 6110 — the first built-in game, Snake.

2002

Palm Treo — phone, PDA and e-mail in one device.

What changed? Better batteries, better coverage and cheaper plans. Manufacturers didn't want to write every app themselves — but they didn't want to open their platforms either.









Proprietary

BEFORE THE APP STORE

The mobile web and the SMS economy

WAP

- Wireless Application Protocol — a simplified HTTP for unreliable mobile networks
- WML instead of HTML, built around “cards”
- Popular WAP sites: CNN, ESPN

Buying via SMS

- Text a premium-rate number → get a ringtone, wallpaper or game
- The carrier controls what's offered and takes a cut
- Every manufacturer had its own OS — hard for third-party apps

Proprietary

QUICK OVERVIEW · 2007 → TODAY

The modern era: platforms, stores, tools



2007
iPhone — multi-touch interface

2008
App Store, Android Market and the first Android phone



2010
iPad popularizes tablets

2014
Material Design — Google's design language

2017
Google announces official Kotlin support

2019
Android goes “Kotlin-first”; Apple introduces SwiftUI

2021
Jetpack Compose 1.0 — declarative UI

2023 →
AI models running on the device (e.g. Gemini Nano)

Dark tiles = the tools we use in this course: Kotlin + Jetpack Compose.

Proprietary

ARCHITECTURE

3-tier architecture of mobile computing

TIER 1

Presentation

The app on the device: UI, input, sensors

Attendance: “Check in” screen, QR code scanning

TIER 2

Application

Business logic and services (API, cloud)

Attendance: validate the code, time and location; sign users in

TIER 3

Data

Storing and accessing data

Attendance: check-ins in a cloud database + a local cache for offline use

Each tier can change independently — the same backend can serve Android, iOS and web clients.

Proprietary

MOBILE ECOSYSTEM

An app lives in an ecosystem, not alone

Operating systems

Android, iOS — and their versions on thousands of devices

Manufacturers and carriers

Samsung, Google, Apple, Xiaomi… networks and plans

App stores

Google Play, App Store — distribution, policies, app review

Cloud services

Firebase, push notifications, maps, payments, AI APIs

Developer tools

IDEs, SDKs, emulators, libraries (Jetpack)

Users and developers

Reviews, community, documentation

THE TWO MAIN OPERATING SYSTEMS

Android and iOS

	Android	iOS
Devices	Thousands of models, every size	iPhone and iPad only
Language	Kotlin (previously Java)	Swift (previously Objective-C)
UI framework	Jetpack Compose	SwiftUI
IDE	Android Studio (Win, Mac, Linux)	Xcode (macOS only)
Openness	Open-source core (AOSP)	Closed, tightly controlled
Distribution	Google Play + other sources	App Store

DEVELOPMENT APPROACHES

Four ways to build an app

Approach	Technologies	Pros	Cons
Native	Kotlin + Compose; Swift + SwiftUI	Best performance, full hardware access	Two separate codebases for two platforms
Cross-platform	Flutter, React Native, Kotlin Multiplatform, .NET MAUI	One codebase (or shared logic) for several platforms	Abstraction layer, slower access to new APIs
Web / PWA	HTML, CSS, JavaScript	No install, runs in the browser	Limited access to sensors and background work
Low-code	App builders, AI generators	Fast prototype without programming	Little control, platform lock-in

Proprietary

CROSS-PLATFORM TODAY

Who stayed, and who's gone

Flutter · Dart

Google; renders its own UI, Hot Reload

React Native · JS/TS

Meta; native components, Fast Refresh

Kotlin Multiplatform · Kotlin

JetBrains; shares logic, UI optional (Compose Multiplatform)

.NET MAUI · C#

Microsoft; successor to Xamarin

⚠ **Discontinued:** Adobe PhoneGap (2020) · Xamarin (support ended May 2024) · Sencha Touch

Proprietary

DEVELOPMENT ENVIRONMENTS

What we need to build apps

IDE and SDK

- **Android Studio** + Android SDK
- Xcode (iOS, macOS only)
- VS Code (Flutter, React Native)
- AI assistants in the IDE (e.g. Gemini in Android Studio)

Where we run apps

- Emulator (Android Virtual Device)
- Physical device via USB or Wi-Fi (ADB)
- Different screen sizes and OS versions

Lab task

- Install Android Studio
- Create an emulator
- Enable Developer options on your phone

FACTORS IN DEVELOPING MOBILE APPS

Mobile is not a “small desktop”

Mobile software engineering

- Fragmentation: many devices and OS versions
- An app lifecycle the OS can interrupt
- Updates go through the store and its review

Device constraints

- Battery, memory, CPU
- Unreliable network — working offline
- Permissions and privacy (camera, location)

User interface

- Small screen, touch input
- Different sizes, rotation, foldable screens
- Accessibility and localization (bs/en)

NEW DIMENSIONS OF MOBILE COMPUTING

What's changing right now

On-device AI

Local models: summaries, recognition, data stays on the phone

AI in development

AI writes code — developers must be able to read, check and fix it

New form factors

Foldables, tablets, watches, cars, TV — adaptive UI

Sensors and IoT

The phone as a hub: health, smart home, BLE devices

Privacy

Stricter permissions, store policies, GDPR

5G and offline-first

Faster networks, but the app must work without them too

Proprietary

OUR CHOICE

Why native Android in this course?

01 **Market:** about three quarters of phones in BiH run Android

02 **Hardware:** camera, location, sensors and notifications with no middle layer

03 **Modern tools:** Kotlin + Jetpack Compose — the official path for Android

04 **Transferable skills:** Kotlin also leads to Kotlin Multiplatform and backend

05 **No Mac needed:** Android Studio runs on Windows, Linux and macOS

Proprietary

ALL SEMESTER LONG

Together, we build an attendance app

Compose UI | QR check-in | Location check | Multiple check-ins per lecture | Firebase | Works offline

Paper sign-in sheets become history — and you learn on a real problem.

Proprietary

BREAK · THEN PART 2

Introduction to Kotlin

Part 1 — language basics

Open play.kotlinlang.org and follow the examples live

Proprietary

KOTLIN

Why Kotlin?

Concise
Less code than Java for the same thing

Safe
Null safety built into the type system

Interoperable
Works alongside Java code and libraries

Android-first
Google's recommended language since 2019

Multiplatform
Android, iOS, backend, web

Tooling
JetBrains builds both the language and the IDE platform

Fun fact: it's named after Kotlin Island in the Gulf of Finland, near St Petersburg. It is also, reportedly, lactose-, sugar- and gluten-free.

BASICS

Hello, world!

- **main** is the entry point — a top-level function, no class needed
- **package** organizes code into logical units
- **print** prints, **println** prints and adds a line break
- No semicolons at the end of lines

Practice: play.kotlinlang.org or a Scratch file in Android Studio

```
// Program entry point
fun main() {
    println("Hello, RMAS!")
}

// Variant with arguments
fun main(args: Array<String>) {
    println("Arguments: ${args.size}")
}
```

Proprietary

VARIABLES

val, var and const val

- **val** — read-only reference
- **var** — mutable
- **const val** — compile-time constant, in UPPER_CASE
- Static typing: the compiler infers the type, and it never changes

Rule: val by default, var only when you must.

```
const val MAX_STUDENTS = 60

fun main() {
  val name = "Sanja"  // String
  var score = 10      // Int
  score = 12          // OK
  // name = "Amra"    // Error:
  // val cannot be reassigned

  var width: Int = 12
  val length: Double = 2.5
}
```

Proprietary

DATA TYPES

Basic types and two traps

Type	Example
Int / Long	42 / 42L
Double / Float	3.14 / 3.14f
Boolean	true, false
Char	'A'
String	"Kotlin"

```
// Trap 1: integer division
1 / 2       // 0
1.0 / 2     // 0.5

// Trap 2: no implicit conversion
val i: Int = 6
// val b: Byte = i  // type mismatch
val b: Byte = i.toByte()

val million = 1_000_000 // readable
```

Proprietary

STRINGS

String templates

- **$name** — inserts a variable's value
- **${expression}** — inserts the result of an expression
- **"""..."""** — multi-line text, no escape sequences

Question: what does the second println print, and why?

```
val dogs = 3
val cats = 2
println("I have $dogs dogs and $cats cats")
// I have 3 dogs and 2 cats

val s = "abc"
println("$s.length is ${s.length}")
// abc.length is 3

val text = """
    Multiple lines
""".trimIndent()
```

Proprietary

CONTROL FLOW

if is an expression

- **if** returns a value — you can assign it to a variable
- That's why Kotlin has no ternary operator **? :**
- Range **1..100** and the **in** operator check membership

```
val guests = 30
val label = if (guests == 0) {
    "No guests"
} else if (guests < 20) {
    "Small group"
} else {
    "Large group"
}
println(label)  // Large group

if (guests in 1..100) println(guests)
```

Proprietary

CONTROL FLOW

when instead of switch

- Branches can be values, ranges or conditions
- Like **if**, **when** can return a value
- Used as an expression, it needs an **else**
- Later: when over sealed classes in Compose UI

```
val results = 50
when (results) {
    0 -> println("No results")
    in 1..39 -> println("Got results!")
    else -> println("That's a lot!")
}

val cost = 450
val discount = when (cost) {
    in 0 until 300 -> 0
    in 300 until 1500 -> 5
    else -> 15
}  // discount = 5
```

Proprietary

CONTROL FLOW

Loops and ranges

- **for … in** — no manual counter
- **..** includes the end, **until** doesn't
- **downTo**, **step**
- **withIndex()** gives index and element
- **while**, **do-while**, **repeat**

```
val pets = listOf("dog", "cat", "canary")
for (p in pets) print("$p ")
for ((i, p) in pets.withIndex()) {
    println("$i: $p")
}

for (i in 1..5) print(i)       // 12345
for (i in 0 until 3) print(i)  // 012
for (i in 5 downTo 1) print(i) // 54321
for (i in 3..6 step 2) print(i)// 35

var n = 0
while (n < 3) n++
repeat(2) { print("Hi! ") }
```

COLLECTIONS

Lists and arrays

- **listOf** — read-only
- **mutableListOf** — add and remove
- **arrayOf** — fixed size, elements mutable
- **val** locks the reference, not the contents

In Android code we mostly use List.

```
val rooms = listOf("A1", "A2", "B3")
// rooms.add("C1")  // Error: read-only

val present = mutableListOf("Amar")
present.add("Lejla")
present.remove("Amar")  // true

val pets = arrayOf("dog", "cat")
pets[0] = "rabbit"
println(pets.contentToString())
// [rabbit, cat]

val all = intArrayOf(1, 2) + intArrayOf(3)
```

Proprietary

FUNCTIONS

fun, default and named arguments

- Keyword **fun**; parameters are typed and read-only
- Single-expression function: **= expression**
- A **default** value makes a parameter optional
- **Named** arguments: clearer calls, any order

```
fun greet(name: String): String {
    return "Hello, $name!"
}
fun square(x: Int) = x * x

// default and named arguments
fun checkIn(
    student: String,
    week: Int = 1,
    late: Boolean = false
) = "$student, week $week, late: $late"

checkIn("Amar")
checkIn("Lejla", late = true)
```

Proprietary

EXERCISE · 5 MINUTES

Four Java functions → one in Kotlin

```java
// Java
String foo(String name, int number, boolean toUpperCase) {
    return (toUpperCase ? name.toUpperCase() : name) + number;
}
String foo(String name, int number) { return foo(name, number, false); }
String foo(String name, boolean toUpperCase) { return foo(name, 42, toUpperCase); }
String foo(String name) { return foo(name, 42); }
```

Write **one** Kotlin function so that every call on the right compiles.

```kotlin
foo("a")                    // a42
foo("b", number = 1)        // b1
foo("c", toUpperCase = true)// C42
```

Proprietary

NULL SAFETY

String and String? are different types

- NullPointerException is the most common crash in Java apps
- In Kotlin, types are **non-null** by default
- **?** after a type allows null
- The compiler forces a check before use — then **smart cast** kicks in

```
val name: String = "Lejla"
// val x: String = null  // Error

val nickname: String? = null
// nickname.length       // Error

if (nickname != null) {
    println(nickname.length)
    // here nickname is a String
} else {
    println("No nickname")
}
```

Proprietary

NULL SAFETY

?. ?: and let

- **?.** safe call — returns null instead of crashing; works in chains
- **?:** Elvis — a fallback if the left side is null; the right side only runs then
- **?.let { }** — run a block only for a non-null value

```
// safe calls in a chain
println(student.group?.assistant?.name)

// Elvis: fallback value
val shown = nickname ?: "No nickname"

// Elvis with return and throw
fun email(id: String): String? {
    val s = findStudent(id) ?: return null
    return s.email ?: throw Exception("…")
}

nickname?.let { println(it) }
```

Proprietary

NULL SAFETY

!! and TODO(): they compile, then crash

- **!!** converts to a non-null type and throws an NPE if the value is null
- **TODO()** always throws NotImplementedError — handy for sketching code

⚠ **!!** in AI-generated code = red flag. Ask: why can't this be null?

```
// “trust me, it's not null”
val len = nickname!!.length
// NullPointerException if null

// compiles, crashes when called
fun findStudent(id: String): Student? =
    TODO("Find student $id")
```

LAMBDA EXPRESSIONS

Functions as values

- Lambda: **{ parameters -> body }**
- A last lambda argument goes **outside the parentheses** (trailing lambda)
- One parameter: implicitly **it**

This is how Compose code looks:
Button(onClick = { ... }) { Text("Check in") }

```
val sum: (Int, Int) -> Int = { x, y -> x + y }
val mul = { x: Int, y: Int -> x * y }
println(sum(2, 3))     // 5

val numbers = listOf(1, 2, 3, 4)
// trailing lambda
numbers.fold(1) { acc, e -> acc * e } // 24

// only argument: no parentheses
run { println("Very cool") }

numbers.filter { it % 2 == 0 } // [2, 4]
```

Proprietary

CODE-READING EXERCISE · IN PAIRS, 7 MINUTES

AI wrote this code. What's wrong with it?

- Find at least **3 problems**
- For each: when does it crash or return a wrong result?
- How would you fix it?

It compiles without errors — which is not the same as correct code.

```
fun average(grades: List<Int>): Int {
    var sum = 0
    for (i in 0..grades.size) {
        sum += grades[i]
    }
    return sum / grades.size
}

fun printStudent(name: String?) {
    println("Student: " + name!!.uppercase())
}
```

SOLUTION

Four problems, one fix

Problem	Consequence
0..grades.size	The range includes size → IndexOutOfBoundsException
sum / grades.size	Integer division: an average of 4.5 becomes 4
Empty list	Once the loop is fixed: division by zero → ArithmeticException
name!!	NullPointerException when the name is unknown

```
fun average(grades: List<Int>): Double =
    if (grades.isEmpty()) 0.0 else grades.sum().toDouble() / grades.size

fun printStudent(name: String?) = println("Student: ${name?.uppercase() ?: "unknown"}")
```

Proprietary

What's next

THIS WEEK'S LAB

Android Studio, emulator, first Compose project and Kotlin exercises

NEXT LECTURE

Kotlin II: classes, data and sealed classes, collections, extensions — then Compose basics

PROJECT

Start thinking about your own app idea — elevator pitch coming soon

RESOURCES

kotlinlang.org/docs · play.kotlinlang.org/byExample · Kotlin Koans · developer.android.com/courses

Thank you!